

Web Bricks v1

02 May 2025 16:24

Got it! Here's a cleaner version of the coding guidelines using only bullet points, maintaining a professional and concise tone:

- Always use specific types in TypeScript instead of using any. For example, use `let data: string = "value"` to ensure type safety.
 - Write both unit and integration tests for key features to ensure functionality and prevent regressions.
 - Manage fixture values exclusively within the fixture files. Avoid modifying fixture data directly inside test files.
 - Test expected values with meaningful data instead of checking only for absence of null or undefined. For example, prefer `toEqual('some value')` over `not.toContain('undefined')`.
 - Use nullish coalescing (`??`) or optional chaining (`?.`) to provide default values instead of relying on logical OR (`||`) in all cases.
 - Use `??` when you want to fallback only for null or undefined.
 - Use `||` only when fallback should apply to all falsy values (e.g., `false`, `0`, `"`, etc.).
 - When updating multiple states sequentially, combine them into a single `updateStateMulti()` call to avoid unnecessary DOM updates.
 - Use fixture data to validate expected results in tests instead of hardcoded values.
 - Always mock API calls with complete and valid scenarios to avoid test warnings or incomplete coverage.
-
- Always import modules using the **full path** to ensure clarity and avoid path resolution issues.
Example:
`import { DISABLE_CLASS, HIDDEN_CLASS } from 'wb-business/portfolio-shared/common/css';`
 - Move expected response objects into the **fixture file** and reuse them in tests instead of redefining them within each test case.
 - Avoid using **magic numbers**. Define constants with meaningful names for better readability and maintainability.
 - If a code block is repeated in multiple places, **create a utility method** and reuse it to avoid duplication and ensure consistency.
 - Do not use `async` in test blocks if there is no `await`. Remove `async` in such cases to keep the code clean and concise.
 - Avoid using `any` for HTML elements. Use appropriate types like `HTMLInputElement` for better type safety and IntelliSense support.
Example:
`const vpNameInput = document.getElementById('newVirtualPortfolioNameModalInput') as HTMLInputElement;`
 - Avoid generic method names like `dispatchWatchlistEventByName`. Use **function parameter names directly** if the method is just dispatching based on input.
Example:
`const dispatchWatchlistEvent = (eventName: string) => window.dispatchEvent(new CustomEvent(eventName, { bubbles: true }));`
 - Use **template literals** for multi-line strings instead of concatenating strings using `+`.